

a macaroon that requires bearers to prove possession of a certain private key. For this, the service can add a first-party structural caveat whose predicate states “the set of macaroons must contain a digitally signed message m , verifiable by the public key P .” The bearer would then be required to present such a signed message along with each request to the target service, if it is to be authorized by a macaroon with this caveat.

H. Public-key-based Macaroon Constructions

Many existing authorization systems are based on public-key certificates, including SPKI/SDSI [16], KeyNote trust management [11], etc. Several of these systems are not truly decentralized, but instead require a globally-accessible infrastructure, such as a revocation list, or a repository of certificates. Further, upon an authorization request, some of these systems carry out a certificate-chain discovery process that can be costly; e.g., the worst-case complexity for SPKI/SDSI being cubic in the size of the repository, as shown in [16].

Instead, authorization based on public-key certificates can benefit from following the patterns of macaroon constructions, as an instance of proof-carrying authorization [6]. Already, some public-key-based applications use a delegation pattern where authority is sent between principals by chaining an existing certificate to a newly-minted public/private key pair—handed off to a new principal—such that the recipient holds a private key whose public key is certified to wield a part of the sender’s authority.

By having each principal, recursively, apply the above pattern to all authorizations, public-key certificates can effect the same type of delegation, attenuation, and key distribution as macaroons. While increasing the number of online messages, following such a macaroon-based pattern eliminates the need to maintain a global certificate repository, since each principal will individually manage the set of certificates relevant to it, and each certificate is unique.

Furthermore, by appropriately choosing the certificate validity times, and the predicates in their assertions, other benefits of macaroons may be replicated. In particular, such a certificate-based construction can provide the same privacy benefits as macaroons, in preventing linkability. Unfortunately, unlike the MAC in macaroons, the cryptographic primitives of public-key certificates can incur significant overhead—as shown in the measurements of Section VII—especially for the minting of the new public/private key pairs essential to the above construction. Therefore, to be more widely applicable, public-key-based macaroon constructions might also make use of HMAC-based signatures, e.g., as in [41].

VI. APPLICATIONS AND USE PATTERNS

Macaroons may be used not only to reimplement existing authorization mechanisms—to state them using a single, unified model, rather than using specific, bespoke protocols—but also to enable new types of authorization for Cloud applications. For example, as in Section V-D, macaroon credentials may be split, so that a separate party holds a necessary caveat discharge, thereby enabling a form of privilege amplification similar to that in [52]. In addition to building such new protocols, macaroons can also be used to enhance several existing authorization mechanisms.

A. Macaroons and End Users

Macaroons are useful for many existing end-user Cloud application scenarios. For example, a macaroon can be minted for sharing an image on Google Docs or Dropbox, with third-party caveats limiting its use to a particular set of users. In such scenarios, when making use of a macaroon, end users need not experience any significant change from the workflow they use today. The client interface of the Cloud service could allow the user to request an “unlisted URL”, with options for selecting the allowed recipients, and provide means for that URL to be shared with those parties, e.g., over email.

Sometimes, a user’s management of macaroon credentials may involve more than one application. For example, when sharing a Dropbox image with a limited group, a user might copy its URL from Dropbox to a service such as Google+ or Facebook, and use that service’s client interface to pick the recipient group and attenuate the authority of the URL’s macaroon credentials. Similarly, when visiting the link for this shared URL, its recipients might be redirected to a login screen, to discharge third-party caveats, before being able to access the image at the URL.

In general, since macaroons are easily serialized, the current mechanisms for sharing and storing access tokens over email, social postings, and chat may continue to work as before. However, instead of copying and pasting URLs, the user interfaces for macaroons may be more user-friendly, e.g., taking the form of explicit, custom “Share” buttons, similar to those used for minting OAuth2 tokens today.

B. Existing Use of Contextual Caveats

Current bearer token authorization relies heavily on implementation guidelines, which puts the security of the mechanisms in the hands of implementors. For example, Mozilla’s BrowserID recommends constraining messages by checking the domain of the relying party [42], and many similar checks apply to OAuth2 [40]. By using macaroons, such guidelines can be made into explicit, mandatory contextual caveats.

As another example, ChannelID binds bearer tokens to a channel-specific public key [7]. ChannelID is well suited for use in first-party caveat predicates that aim to contextually confine the use of macaroons to specific (recipient) principals.

Such caveats are likely to use application-specific predicate languages, similar to how Bucket Policies in the Amazon S3 service define a rich policy language for contextual authorization of clients accessing S3 storage [5]. Of course, using macaroons, such policies could be set per issued token, instead of per storage bucket, efficiently and with very little effort.

C. Cookies as Macaroons

Cookies are widely used for authorization bearer tokens, such as HTTP session identifiers. With macaroon-based cookies, immediate benefits may be gained: macaroons would ensure integrity despite not maintaining state at the server, and, through the use of caveats, allow the cookies to be bound to specific, observable client context, such as IP address, user agent strings, referrers etc. An interesting benefit is also that clients could add caveats, for example to limit the usefulness of stolen cookies. Even simple attenuation might offer clients